

CloudSound - Končno Poročilo Projekta

Člana skupine: Tevž Sedmak in Matjaž Kumin

Številka projektne skupine: 23

1. Dostop do Projekta

Povezava do aplikacije: <http://9.235.118.168/>

Testni vpisni podatki (admin): - Uporabniško ime: `admin@test.com` - Geslo: `admin123`

GitHub Organizacija: <https://github.com/CloudSound-MKNZ>

GitHub Repozitoriji: - <https://github.com/CloudSound-MKNZ/CloudSound.git> (glavna dokumentacija) - <https://github.com/CloudSound-MKNZ/cloudsound-shared.git> (skupne knjižnice) - <https://github.com/CloudSound-MKNZ/cloudsound-musicdiscovery.git> - <https://github.com/CloudSound-MKNZ/cloudsound-event-manager.git> - <https://github.com/CloudSound-MKNZ/cloudsound-concertmanagement.git> - <https://github.com/CloudSound-MKNZ/cloudsound-radiostreaming.git> - <https://github.com/CloudSound-MKNZ/cloudsound-authentication.git> - <https://github.com/CloudSound-MKNZ/cloudsound-api-gateway.git> - <https://github.com/CloudSound-MKNZ/cloudsound-analytics.git> - <https://github.com/CloudSound-MKNZ/cloudsound-admin-management.git>

2. Kratek Opis Projekta

CloudSound je spletna aplikacija, namenjena lokalnemu glasbenemu klubu za upravljanje napovednika koncertov in hkrati ponuja uporabnikom možnost poslušanja glasbe preko integriranega radia. Sistem rešuje problem avtomatizacije upravljanja koncertov in zagotavljanja dostopa do glasbe prihajajočih in preteklih izvajalcev. Aplikacija avtomatsko pridobiva glasbo iz YouTube in Bandcamp API-jev ter omogoča poslušanje različnih radijev, deljenih po žanrih, preteklih izvajalcih in prihajajočih koncertih. Sistem samodejno povezuje Facebook dogodke z napovednikom koncertov in uporabnikom ponuja preprosto izkušnjo odkrivanja glasbe.

3. Ogrodje in Razvojno Okolje

Backend Tehnologije

- **Programski jezik:** Python 3.11+
- **Mikrostoritveno ogrodje:** FastAPI
- **Podatkovna baza:** PostgreSQL 15
- **Message Queuing:** Kafka (event streaming), RabbitMQ (task queues)
- **Predpomnjenje:** Redis

Frontend Tehnologije

- **Ogrodje:** SvelteKit
- **Styling:** TailwindCSS

Oblak in Infrastruktura

- **Kontejnerizacija:** Docker
- **Orkestracija:** Kubernetes (K3s lokalno, AKS na Azure)
- **Cloud Platform:** Microsoft Azure
- **Infrastructure as Code:** Terraform
- **CI/CD:** GitHub Actions (ne dela zaradi pomanjkanja dovoljenja na Azure)

- **Paketiranje:** Helm Charts

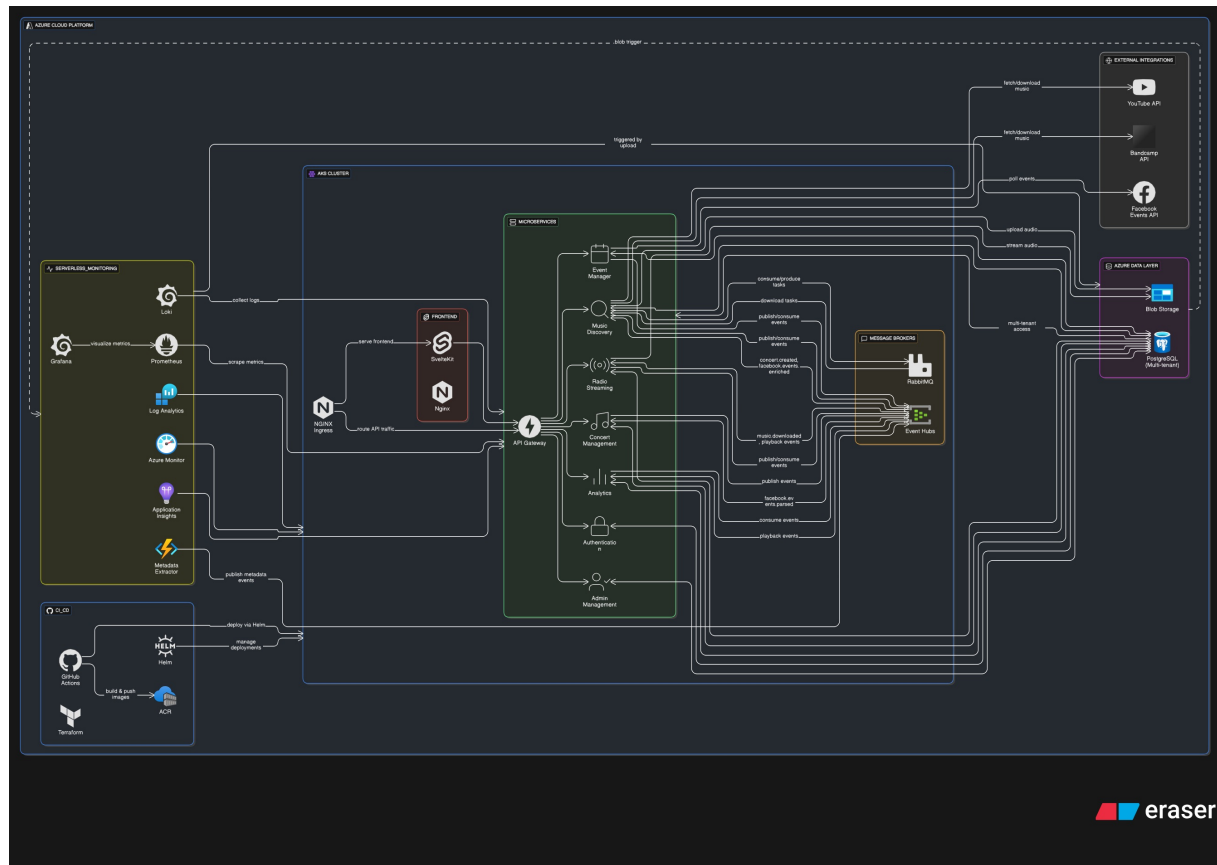
Zunanje Integracije

- **API-ji:** YouTube Data API v3, Bandcamp API, Facebook Graph API
- **Shranjevanje datotek:** MinIO (S3-compatible) / Azure Blob Storage
- **Avtentikacija:** JWT (JSON Web Tokens)
- **Komunikacijski protokoli:** REST API, gRPC, WebSockets (streaming)

Monitoring in Opazovanje

- **Metrike:** Prometheus
- **Vizualizacija:** Grafana
- **Beleženje:** Structlog (strukturirano JSON beleženje)
- **Sledenje:** CorrelationID middleware

4. Shema Arhitekture



Arhitektura sistema CloudSound

Slika 1: Arhitektura sistema CloudSound

Shema prikazuje:

- 8 mikrosoritev (API Gateway, Authentication, Admin Management, Concert Management, Event Manager, Music Discovery, Radio Streaming, Analytics)
- PostgreSQL podatkovne baze (per-service ali shared)
- Kafka in RabbitMQ message brokerje
- Redis cache
- MinIO/Azure Blob Storage za MP3 datoteke
- Ingress Controller (NGINX)

- Zunanje API-je (YouTube, Bandcamp, Facebook)
 - Frontend (SvelteKit)
 - Komunikacijske protokole (REST, gRPC, Kafka events)
 - Azure Serverless Functions
-

5. Seznam Funkcionalnosti Mikrostoritev

5.1 API Gateway (cloudsound-api-gateway)

- Centralna vstopna točka za vse zunanje zahteve (port 8000)
- Usmerjanje prometa do ustreznih backend mikrostoritev
- JWT avtentikacija preko AuthMiddleware
- Rate limiting za preprečevanje zlorab (RateLimitMiddleware)
- CORS upravljanje
- Request/Response logiranje z correlation ID
- Proxy middleware za transparentno posredovanje zahtev

5.2 Authentication Service (cloudsound-authentication)

- Registracija in prijava uporabnikov (admin profili)
- Izdaja in osvežitev JWT tokenov (access & refresh tokens)
- Validacija JWT tokenov
- Role-based access control (admin/user vloge)
- Password hashing z bcrypt
- Token revocation mehanizem

5.3 Concert Management Service (cloudsound-concertmanagement)

- CRUD operacije za koncerte (dodajanje, urejanje, brisanje)
- Upravljanje sezon koncertov
- Povezovanje izvajalcev z dogodki
- Shranjevanje metapodatkov koncertov (datum, lokacija, žanr, cena)
- Objava "concert.created" dogodkov na Kafka
- Integracija z YouTube/Bandcamp linki v opisu

5.4 Event Manager Service (cloudsound-event-manager)

- Avtomatska integracija s Facebook Graph API
- Periodično pridobivanje Facebook dogodkov (scheduled job)
- Mapiranje zunanjih dogodkov na interne koncerte
- Sinhronizacija metapodatkov (naslov, opis, čas)
- Circuit breaker zaščita za zunanje API klice
- Retry logika z eksponentnim backoff-om

5.5 Music Discovery Service (cloudsound-musicdiscovery)

- Avtomatsko iskanje in prenos glasbe preko YouTube API
- Integracija z Bandcamp API za dodatne glasbene vire
- Ekstrakcija audio from YouTube videov (yt-dlp)
- Shranjevanje MP3 datotek v MinIO/Azure Blob Storage
- Metadata extraction (izvajalec, naslov, album, žanr)
- RabbitMQ task queue za asinhrono procesiranje prenosov
- Ustvarjanje "music.downloaded" dogodkov na Kafka
- Priporočilni sistem za podobne izvajalce

5.6 Radio Streaming Service (cloudsound-radiostreaming)

- Upravljanje več radijskih postaj (prihajajoči, pretekli, žanri)
- Playlist management z različnimi strategijami
- Audio streaming z crossfade prehodi med skladbami
- WebSocket povezave za real-time playback control
- Zgodovina poslušanja uporabnikov
- Playback event logging za analitiko
- Support za več sočasnih streamov

5.7 Admin Management Service (cloudsound-admin-management)

- Admin dashboard za upravljanje sistema
- Pregled in upravljanje uporabnikov
- Statistika uporabe radia (globalna, ne per-user)
- Agregacija podatkov iz različnih servisov
- Admin-only endpoints z role-based zaščito

5.8 Analytics Service (cloudsound-analytics)

- Zbiranje playback dogodkov iz Kafka
 - Agregacija statistike poslušanja (najbolj predvajane skladbe, žanri)
 - Časovna analiza uporabe (peak hours)
 - Export podatkov za poročanje
 - Real-time dashboard metrics
 - Dolgoročno shranjevanje zgodovinskih podatkov
-

6. Primeri Uporabe

6.1 Osnovni Primeri Uporabe (Uporabnik)

1. **Pregled prihajajočih koncertov** - Uporabnik obišče spletno stran in si ogleda napovednik prihajajočih koncertov z vsemi podrobnostmi (datum, lokacija, izvajalec, povezava na Facebook event).
2. **Poslušanje radia "Prihajajoči bendi"** - Uporabnik izbere radio postajo, ki predvaja glasbo bendov, ki bodo kmalu nastopali v klubu.
3. **Poslušanje radia "Pretekli bendi"** - Uporabnik posluša glasbo izvajalcev, ki so že nastopali v klubu.
4. **Poslušanje radia po žanru** - Uporabnik izbere specifičen žanr (rock, jazz, metal, electronic) in posluša ustrezno glasbo.
5. **Iskanje glasbe** - Uporabnik išče glasbo po imenu izvajalca ali skladbe skozi integrirano iskalno funkcijo.
6. **Pregled zgodovine poslušanja** - Uporabnik si ogleda zgodovino predvajanih skladb na izbranem radiu.

6.2 Admin Primeri Uporabe

7. **Admin registracija** - Samo uporabniki z admin pravicami lahko registrirajo nove admin profile.
8. **Dodajanje koncerta** - Organizator (admin) doda nov koncert z vsemi potrebnimi podatki (datum, čas, lokacija, izvajalec, žanr, cena, YouTube/Bandcamp link).
9. **Urejanje koncerta** - Organizator posodobi informacije o obstoječem koncertu.
10. **Brisanje koncerta** - Organizator odstrani koncert iz napovednika.
11. **Upravljanje sezone** - Organizator načrtuje in organizira koncerte za prihajajočo sezono.

12. **Facebook integracija** - Po dodajanju koncerta sistem avtomatsko poveže Facebook event z napovednikom na podlagi datuma in imena izvajalca.

6.3 Kompleksen Primer Uporabe: Dodajanje Novega Koncerta (Multi-Service Workflow)

Scenarij: Admin doda nov koncert za bend "The Midnight" z datumom 15. marec 2026.

Potek:

1. **Concert Management Service:**

- Admin pošlje POST zahtevo z podatki o koncertu preko frontend-a
- API Gateway validira JWT token in posreduje zahtevo
- Concert Management ustvari nov zapis v PostgreSQL bazi
- Service objavi "concert.created" event na Kafka topic

2. **Event Manager Service:**

- Posluša "concert.created" events na Kafka
- Samodejno poišče ustrezne Facebook events za "The Midnight" okoli datuma 15.3.2026
- Če najde ujemanje, poveže Facebook event ID z internim koncertom
- Sinhronizira dodatne podatke (cover photo, opis)

3. **Music Discovery Service:**

- Posluša "concert.created" events na Kafka
- Avtomatsko sproži iskanje glasbe za "The Midnight" na YouTube in Bandcamp
- Doda naloge v RabbitMQ queue za prenos najboljših skladb
- Worker procesi prenesejo MP3 datoteke in jih shranijo v MinIO/Azure Blob
- Po uspešnem prenosu objavi "music.downloaded" events na Kafka
- Azure Serverless Function ekstraktira metadata iz MP3 datotek

4. **Radio Streaming Service:**

- Posluša "music.downloaded" events na Kafka
- Doda nove skladbe v playlist za radio "Prihajajoči bendi"
- Posodobi interno playback strategijo

5. **Analytics Service:**

- Zabeleži dogodek dodajanja koncerta za statistiko
- Pripravi dashboard metrike za admin pogled

Sodelujoče mikrororitve: 6 (API Gateway, Concert Management, Event Manager, Music Discovery, Radio Streaming, Analytics)

Komunikacijski protokoli: REST (frontend → API Gateway), gRPC (interne komunikacije), Kafka (event streaming), RabbitMQ (task queue)

7. Seznam Opravljenih/Vključenih Projektnih Zahtev

7.1 Repozitorij

Projekt uporablja Git z multi-repository pristopom na GitHub platformi znotraj organizacije CloudSound-MKNZ. Vsak servis ima svoj ločen repozitorij z README.md dokumentacijo, Dockerfile za kontejnerizacijo in requirements.txt za Python odvisnosti. Skupne knjižnice so izdvojene v cloudsound-shared repozitorij, ki ga druge mikrororitve uporabljajo kot dependency. Vsi repozitoriji so javno dostopni.

Dostop: <https://github.com/CloudSound-MKNZ>

7.2 Mikrororitve in "Cloud-Native" Aplikacija

Sistem je sestavljen iz 8 neodvisnih mikrororitev: api-gateway, authentication, admin-management, concert-management, event-manager, music-discovery, radio-streaming in analytics. Vsaka mikrororitev je samostojno deployljiva v Docker kontejnerju in ima svojo PostgreSQL bazo oz. shema. Komunikacija med servisi poteka

preko REST API-jev, gRPC za interne klice ter Kafka/RabbitMQ za asinhrono sporočanje. Kubernetes manifesti in Helm charts so organizirani v `infrastructure/helm/` direktoriju. Podatkovne migracije so centralizirane v `cloudsound-shared/db/migrations/` z uporabo Alembic orodja.

Azure dostop:

- **AKS Cluster:** Azure Portal → Kubernetes services → `cloudsound-aks`
 - **Pregled podov:** Lahko dostopate preko `kubectl` povezanega na AKS cluster ali preko Azure Portal → Workloads
 - **Servisi:** Vsak servis je izpostavljen kot Kubernetes Service, dostopen preko Ingress Controller-ja
-

7.3 Dokumentacija

Celovita projektna dokumentacija se nahaja v `docs/` mapi glavnega CloudSound repozitorija. Vsaka mikrorstitev ima svoj `README.md` z specifičnimi navodili za zagon, testiranje in konfiguracijo.

Dostop:

- GitHub: <https://github.com/CloudSound-MKNZ/CloudSound/tree/main/docs>
 - Lokalno: `docs/` direktorij v kloniranem repozitoriju
-

7.4 Namestitev v Oblak

Projekt je deployiran na Microsoft Azure z uporabo Terraform za Infrastructure as Code. Terraform konfiguracija upravlja naslednje Azure resurse: AKS (Azure Kubernetes Service) cluster, Azure Database for PostgreSQL Flexible Server, Azure Storage Account za blob storage, Azure Event Hubs (kot Kafka replacement), ter pripadajoče networking komponente (VNet, Subnets, NSG). Deployment proces je avtomatiziran preko GitHub Actions workflow-a definiranega v `.github/workflows/deploy-to-azure.yml`. CI/CD pipeline vključuje korake za build Docker slik, push v Azure Container Registry (ACR) ter Helm deployment na AKS cluster.

Ker nam Azure Student Account ne omogoča vzpostavitve resource principal preko Microsoft Entra ID, je CI/CD le zasnovan in ne deployan.

Azure dostop:

- **Resource Group:** Azure Portal → Resource groups → `cloudsound-rg`
- **AKS Cluster:** Azure Portal → Kubernetes services → `cloudsound-aks`
- **PostgreSQL:** Azure Portal → Azure Database for PostgreSQL flexible servers → `cloudsound-db`
 - Connection string dostopen v Key Vault ali environment variables
- **Storage Account:** Azure Portal → Storage accounts → `cloudsoundstorage`
 - Blob containers za MP3 datoteke
- **Terraform State:** Shranjen v Azure Storage Account (backend configuration)

Omejitev: Ker nam Azure Student Account ne omogoča vzpostavitve resource principal preko Microsoft Entra ID, je CI/CD le zasnovan in ne deployan. Zaradi omejitev študentskega Azure računa (quota limits, spending caps) je continuous deployment delno omejen. Production deployment zahteva manualno odobritev.

7.5 Dokumentacija API

Vse mikrorstitev uporabljajo FastAPI framework, ki avtomatsko generira interaktivno OpenAPI (Swagger) dokumentacijo. Za vsak servis je dokumentacija dostopna na `/docs` endpoint-u (Swagger UI) za interaktivno testiranje API klicev ter na `/redoc` endpoint-u (ReDoc) za branje-prijazno različico. OpenAPI 3.0 JSON schema je dostopna na `/openapi.json` endpoint-u in lahko služi za avtomatsko generacijo klientskih SDK-jev. Vsi endpoint-i so dokumentirani z opisi, request/response schemi, primeri in error kodi.

Dostop (lokalno preko port-forward ali Ingress):

- API Gateway: <http://9.235.118.168/docs>
- Concert Management: <http://9.235.118.168/concerts/docs>
- Authentication: <http://9.235.118.168/auth/docs>
- Ostale mikrostoritve: <http://{{base-url}}/{{service-path}}/docs>

Azure dostop:

- Preko Ingress Controller-ja na javnem IP naslovu AKS Load Balancer-ja
- Ingress rules definirajo path-based routing do posameznih servisov

7.6 Cevovod CI/CD

Projekt uporablja GitHub Actions za popolno avtomatizirano CI/CD pipeline-o. Workflow datoteke v `.github/workflows/` vključujejo: `build-images.yml` (gradnja Docker slik za vse servise in push v GitHub Container Registry oz. Azure Container Registry), `ci.yml` (avtomatsko testiranje z pytest, linting s flake8/black, security scanning), `deploy.yml` (deployment na lokalni K3s), `deploy-to-azure.yml` (deployment na Azure AKS z Helm), ter `terraform.yml` (plan in apply Terraform sprememb). Pipeline se sproži ob push-u na main branch ali pull request-ih. Vsaka Docker slika je tagged z Git commit SHA in "latest" tag-om za sledljivost verzij.

Dostop:

- **GitHub Actions:** <https://github.com/CloudSound-MKNZ/{repo}/actions>
- **Container Registry:** GitHub Packages → <https://github.com/orgs/CloudSound-MKNZ/packages>
- **Azure Container Registry:** Azure Portal → Container registries → `cloudsoundacr`
 - Images dostopne v Repositories sekciji
- **Workflow Runs:** Zgodovina vseh runs-ov z logi dostopna v GitHub Actions tab-u

Opomba: Zaradi Azure študentskih omejitev, je production deployment ročno trigger-an (`workflow_dispatch` event).

7.7 Helm Charts

Kubernetes deployments so parametrizirani z uporabo Helm package manager-ja. Helm charts se nahajajo v `infrastructure/helm/cloudsound/` direktoriju z glavno `values.yaml` datoteko za konfiguracijo. Chart vključuje Kubernetes templates za: Deployments (definicije podov, resource limits), Services (ClusterIP za interno komunikacijo), Ingress (external routing z NGINX annotations), ConfigMaps (non-sensitive konfiguracija) in Secrets (sensitive podatki). Podpora za različna okolja (development, staging, production) je implementirana z ločenimi values datotekami (`values-dev.yaml`, `values-prod.yaml`). Chart dependency management vključuje pod-charts za PostgreSQL, Redis in RabbitMQ.

Dostop:

- **Lokalno:** `infrastructure/helm/cloudsound/`
- **Instalirani releases:** `helm list -n cloudsound` (na AKS z ustreznim kubeconfig)
- **Upgrade:** `helm upgrade cloudsound ./infrastructure/helm/cloudsound -f values-prod.yaml -n cloudsound`

Azure dostop: - Helm releases so vidni v AKS cluster-ju pod namespace-om `cloudsound` - Azure Portal → AKS → Workloads za pregled deployanih podov iz Helm chart-a

7.8 "Serverless" Funkcija

Azure Functions so uspešno deployirane v `cloudsound-functions` Function App na Azure. Ključna funkcija `metadata_extractor` je blob-triggered serverless funkcija, ki se sproži ob uploadu MP3 datoteke v Azure Blob Storage `music/` container. Funkcija ekstrahira ID3 metadata (artist, title, album, genre, duration) z uporabo mutagen knjižnice in objavi dogodke na Kafka (Azure Event Hubs). Funkcija je napisana v Python 3.11 z Azure Functions v4 runtime na **Basic (B1) Service Plan**. Konfiguracija je definirana v `function.json` in `host.json`

datotekah. Unit testi so v `test_local.py`, integration testi v `test_k3s.py`.

Deployment Status:

- **Function App:** `cloudsound-functions` (B1 plan - ~13 EUR/month)
- **Function:** `metadata_extractor` (blob-triggered)
- **Region:** Italy North
- **URL:** `https://cloudsound-functions.azurewebsites.net`
- **Trigger:** Blob upload v `music/` container
- **Output:** Metadata JSON v `metadata/` container + Kafka event

Azure Portal Dostop:

- **Function App:** Azure Portal → Function App → `cloudsound-functions`
- **Resource Group:** `cloudsound-rg`
- **Funkcija:** Pod Function App → Functions → `metadata_extractor`
- **Monitoring:** Application Insights (`cloudsound-insights`)
 - Metrics: Executions, Duration, Failures
 - Live Logs: Portal → Function App → Log stream
- **Blob Containers:**
 - Input: `cloudsoundstoragemgo77h` → `music/`
 - Output: `cloudsoundstoragemgo77h` → `metadata/`

Testing:

```
# Upload test MP3 to trigger function
az storage blob upload \
  --account-name cloudsoundstoragemgo77h \
  --container-name music \
  --file test.mp3 \
  --name test-song.mp3

# Check if metadata was extracted
az storage blob list \
  --account-name cloudsoundstoragemgo77h \
  --container-name metadata \
  --query "[?contains(name, 'test-song')]"
```

Opomba: Uporablja B1 plan namesto Y1 Consumption Plan, ker Italy North region ne podpira Consumption Plan-a.

7.9 Zunanji API-ji

Projekt integrira tri zunanje API-je: **YouTube Data API v3** (`youtube_client.py`) za iskanje in pridobivanje video informacij, **Bandcamp API** (`bandcamp_client.py`) za dodatne glasbene vire, in **Facebook Graph API** (`facebook_client.py`) za sinhronizacijo Facebook dogodkov. Vsak API klient implementira circuit breaker pattern (z uporabo `CircuitBreaker` razreda) za zaščito pred nedostopnostjo zunanjih sistemov ter retry logiko z eksponentnim backoff-om (dekorator `retry_with_backoff`). Za development in testing so na voljo mock implementacije v `mock_apis.py`, ki simulirajo odgovore zunanjih API-jev brez porabe kvot.

Konfiguracija:

- API ključi so shranjeni v Azure Key Vault
- Environment variables: `YOUTUBE_API_KEY`, `FACEBOOK_ACCESS_TOKEN`, `BANDCAMP_API_KEY`

Azure dostop do secrets:

- Azure Portal → Key Vault → `cloudsound-keyvault`
- Secrets: Seznam vseh API ključev
- Access policies: Konfigurirane za AKS managed identity

Monitoring:

- Prometheus metriki za eksterne API klice: `external_api_requests_total`, `external_api_errors_total`, `circuit_breaker_state`
 - Dostop do metrik: `http://{service-url}/metrics` (format: Prometheus)
-

7.10 Večnajemništvo (Multitenancy)

Multi-tenant arhitektura je implementirana v `cloudsound_shared/multitenancy/` modulu s podporo za tri izolacijske strategije: **row-level** (shared schema z `tenant_id` kolumno), **schema-level** (ločene sheme per tenant v isti bazi) in **database-level** (popolnoma ločene baze per tenant). `TenantMixin` razred razširja SQLAlchemy modele z tenant awareness funkcionalnostjo. `TenantMiddleware` ekstraktira tenant identifikator iz JWT tokena ali HTTP header-ja (`X-Tenant-ID`) in ga shrani v request context. `TenantAwareSession` SQLAlchemy session razred avtomatsko filtrira vse queries z aktualnim `tenant_id`-jem ter preprečuje cross-tenant data leakage. Implementacija omogoča enostaven prehod med strategijami preko konfiguracije.

Azure implementacija:

- Za production: Schema-level isolation v Azure PostgreSQL Flexible Server
- Vsak tenant ima svoj schema namespace
- Connection pooling z tenant routing logiko

Testiranje:

- Multi-tenant unit testi v `cloudsound_shared/tests/test_multitenancy.py`
 - Smoke tests: `pytest -m multitenancy`
-

7.11 Preverjanje Zdravja (Health Checks)

Health check endpoints so implementirani v `cloudsound_shared/health/` modulu in vključeni v vse mikrororitve. Osnovni `/health` endpoint vrne HTTP 200 če je servis aktiven. `/health/ready` (readiness probe) preverja odvisnosti (database connection, cache connectivity, message broker) in vrne 200 samo če je servis pripravljen sprejemati zahteve. `/health/live` (liveness probe) preverja ali servis proces deluje in ni v deadlock stanju. Kubernetes uporablja te endpoint-e za avtomatski restart unhealthy podov in routing prometa samo na ready instance. Health check responses vključujejo dodatne diagnostične informacije (verzija, uptime, dependency status).

Azure dostop:

- **Kubernetes probes:** Definirane v Helm chart-ih:

```
livenessProbe:
  httpGet:
    path: /health/live
    port: 8080
readinessProbe:
  httpGet:
    path: /health/ready
    port: 8080
```

- **Testiranje:** `curl http://{service-url}/health` ali preko Azure Portal → AKS → Workloads → Pod details → Events
 - **Monitoring:** Health check failures vidni v Prometheus metrikah (`health_check_failures_total`)
-

7.12 GraphQL in gRPC

gRPC je implementiran za komunikacijo med backend mikrororitvami, kar omogoča hitrejšo in bolj type-safe komunikacijo od REST API-jev. Protocol buffer definicije (`.proto` datoteke) so v `cloudsound_shared/protos/` direktoriju. Generirane Python stubs se avtomatsko build-ajo med deploy procesom. Primer: Concert Management Service izpostavlja gRPC endpoints za hitro poizvedovanje koncertov, ki jih uporablja Radio

Streaming Service brez overhead-a HTTP. gRPC streams so uporabljeni za real-time notifikacije. Dokumentacija je v docs/GRPC_IMPLEMENTATION.md in README datotekah posameznih servisov.

GraphQL bi lahko bil implementiran kot dodatna query layer preko API Gateway-ja (ni v trenutni verziji), kar bi frontend-u omogočilo fleksibilnejše queries z manj network roundtrips.

Azure specifika:

- gRPC komunikacija poteka interno v AKS cluster-ju (ClusterIP services)
 - gRPC health checking protokol uporabljen za Kubernetes health probes
 - TLS/mTLS za secure gRPC channels
-

7.13 Sporočilni Sistemi (Message Queues)

Projekt uporablja dva message broker sistema: **Apache Kafka** za event-driven komunikacijo med mikrostoritvami (pub/sub model) in **RabbitMQ** za task queue pattern. Kafka topics: `concerts.created`, `concerts.updated`, `concerts.deleted`, `music.downloaded`, `playback.events`. Event-driven workflow omogoča loose coupling in eventual consistency. Primer: Ko Concert Management objavi `concert.created` event, ga poslušata Music Discovery (sproži download glasbe) in Event Manager (poveže Facebook event). RabbitMQ je uporabljen v Music Discovery servisu za `music.downloads` queue, kjer worker procesi asinkrono procesirajo download opravila z možnostjo retry in dead-letter queue za failed tasks.

Azure implementacija:

- **Kafka Alternative:** Azure Event Hubs (Kafka-compatible endpoint)
 - Azure Portal → Event Hubs Namespaces → `cloudsound-events`
 - Topics vidni pod "Event Hubs" v namespace-u
 - Connection string dostopen v "Shared access policies"
- **RabbitMQ:** Deployiran kot StatefulSet v AKS cluster-ju ali Azure Service Bus za managed rešitev
 - Management UI: `kubectl port-forward svc/rabbitmq 15672:15672`
 - Queues vidne v management console

Monitoring:

- **Kafka metrics:** Consumer lag, messages per second → Prometheus → Grafana dashboard
 - **RabbitMQ metrics:** Queue depth, processing rate → accessible preko management API
-

7.14 Centralizirano Beleženje Dnevnikov (Centralized Logging)

Vsaka mikrostoritev uporablja **structlog** knjižnico za strukturirano beleženje v JSON formatu. Logi vključujejo polja: timestamp, log level, service name, correlation ID, message, context variables. V production modu se vsi logi pišejo v stdout/stderr in jih Kubernetes avtomatsko zajema. `CorrelationIDMiddleware` generira unikaten ID za vsak inbound request in ga propagira skozi vse downstream service klice (preko HTTP header-ja in Kafka event metadata), kar omogoča end-to-end tracing celotnega request flow-a skozi distribuiran sistem. Logi so agregirani v centraliziran logging stack.

Azure dostop:

- **Container Insights:** Azure Portal → AKS → Monitoring → Insights
 - Live Logs: Prikaz real-time logov iz podov
 - Log Analytics: `kubectl logs` alternative z query capabilities
- **Log Analytics Workspace:** Azure Portal → Log Analytics workspaces → `cloudsound-logs`
 - Kusto Query Language (KQL) za napredne queries
 - Primer query: `ContainerLog | where ContainerName contains "concert-management" | where TimeGenerated > ago(1h)`
- **Application Insights:** Povezan za application-level tracing
 - End-to-end transaction tracing z correlation ID

- Dostop: Azure Portal → Application Insights → cloudsound-appinsights → Transaction search

Lokalno testiranje:

- Logi dostopni preko `kubectl logs <pod-name> -f` ali Docker logs
-

7.15 Zbiranje Metrik (Metrics Collection)

Vsaka mikrorstitev izpostavlja Prometheus-kompatibilne metrike na `/metrics` HTTP endpoint-u. Definirane so standardne metrike: **HTTP metrics** (request count, latency histogram, error rate po endpoint-ih), **Kafka metrics** (messages published/consumed, processing latency), **RabbitMQ metrics** (tasks enqueued/processed), ter **business metrics** specifične za servis (npr. `music_downloads_total`, `playback_events_total`, `concerts_created_total`). Metrike so definirane v servisnem `__init__.py` z uporabo `prometheus_client` knjižnice (Counter, Gauge, Histogram instrumenti). Prometheus server periodično scrape-a vse `/metrics` endpoint-e in shranjuje time-series podatke. Grafana dashboardi vizualizirajo metrike za operacionalni monitoring.

Azure dostop:

- **Prometheus:** Deployiran v AKS kot StatefulSet ali Azure Monitor managed Prometheus
 - Azure Portal → Azure Monitor → Managed Prometheus → cloudsound-prometheus
 - Query interface: Prometheus UI dostopen preko Ingress ali port-forward
 - Primer: `kubectl port-forward svc/prometheus 9090:9090`
- **Grafana:**
 - Azure Portal → Azure Managed Grafana → cloudsound-grafana
 - URL: `https://cloudsound-grafana-{hash}.weu.grafana.azure.com`
 - Pre-built dashboards: "CloudSound Overview", "Service Health", "Business Metrics"
- **Metrike posameznega servisa:** `curl http://{service-url}/metrics`
- **Azure Monitor Integration:** Metrike lahko exportamo tudi v Azure Monitor Metrics
 - Azure Portal → Monitor → Metrics → Scope: AKS cluster → Metric Namespace: Prometheus

Ključne metrike za monitoring: - Request rate, error rate, duration (RED metrics) - CPU, Memory, Disk utilization - Database connection pool usage - Message queue depth

7.16 Izolacija in Toleranca Napak (Fault Tolerance)

Implementiran je **circuit breaker** vzorec (pattern) preko `CircuitBreaker` razreda v `music-discovery` in `event-manager` servisih za zaščito klicev zunanjih API-jev (YouTube, Facebook, Bandcamp). Circuit breaker prehaja med stanji: CLOSED (normalno delovanje), OPEN (vse requests takoj failajo brez klica), HALF_OPEN (testni requests za recovery check). Prag je nastavljen na 5 consecutive failures z 60s cooldown periodom. **Retry logika** je implementirana z `retry_with_backoff` dekoratorjem, ki izvede do 3 retries z eksponentnim backoff-om (1s, 2s, 4s). Dodatno so implementirani timeouts na vseh HTTP in gRPC klicih ter graceful degradation (sistem deluje v omejenem načinu, če so dependenci nedostopni).

Azure resilience features:

- **AKS Node pools:** Multi-zone deployment za high availability
 - Azure Portal → AKS → Node pools → Availability zones
- **Pod Disruption Budgets:** Preprečijo simultano terminacijo kritičnih podov
- **Autoscaling:** Horizontal Pod Autoscaler (HPA) za dinamično skaliranje
 - Definirano v Helm charts glede na CPU/Memory metrike
- **Health checks:** Automatic pod restart on failures

Monitoring fault tolerance:

- Circuit breaker state metrika: `circuit_breaker_state{service="youtube"}`
- Retry metrike: `retry_attempts_total`, `retry_success_total`
- Dostop: Prometheus/Grafana → "Fault Tolerance" dashboard

7.17 Upravljanje s Konfiguracijo (Configuration Management)

Konfiguracija je centralizirana v `settings.py` Python modulu z uporabo **pydantic-settings** knjižnice. Settings razredi definirajo tipizirana configuration polja, ki se berejo iz okoljskih spremenljivk in `.env` datotek. Multi-environment support: različne konfiguracije za development, test in production okolja, izbrano preko ENV environment variable. Brez potrebe po re-compile ali rebuild slik za spremembe konfiguracije. Sensitive podatki (database passwords, API keys) so injektirani preko Kubernetes Secrets, ki so referencirani kot environment variables v pod-ih. Non-sensitive konfiguracija je v ConfigMaps. Validation logika v pydantic prepreči runtime napake zaradi manjkajočih ali napačnih konfiguracij.

Azure konfiguracija:

- **Azure Key Vault:** Za sensitive secrets
 - Azure Portal → Key Vault → cloudsound-keyvault → Secrets
 - Injektirano v pods preko CSI driver ali environment variables
- **Kubernetes Secrets:**
 - Create: `kubectl create secret generic cloudsound-secrets --from-literal=db-password=xxx`
 - View: `kubectl get secrets -n cloudsound`
- **ConfigMaps:**
 - Definicija v Helm charts: `templates/configmap.yaml`
 - View: `kubectl get configmaps -n cloudsound`
- **Environment-specific values:** Helm `values-prod.yaml` za production overrides

Best practices:

- Nikoli commit-aj secrets v Git
 - Uporablaj Azure Key Vault references v deployment files
 - Rotation: Pravilen password rotation omogoča Key Vault
-

7.18 Grafični Vmesnik (Frontend)

Frontend je implementiran v **SvelteKit** ogrodju (JavaScript framework), kar omogoča server-side rendering (SSR) in optimalne performance. Komunicira z backend mikrosoritivami preko REST API-jev skozi API Gateway (centralna vstopna točka na port 8000). Vključuje več podstrani: **Radio** (player interface z izbiro postaj, playback controls, currently playing info), **Koncerti** (napovednik s filtri po datumu in žanru), **Admin Panel** (dashboard za upravljanje koncertov, statistika), **Iskanje** (search po glasbenikih in skladbah). UI je responsive z TailwindCSS styling frameworkom, kar omogoča optimalen prikaz na mobilnih napravah in desktopih. WebSocket povezave za real-time audio streaming in live updates.

Azure deployment:

- Frontend je static build deployiran kot:
 1. **Azure Static Web Apps** (opcija 1) - serverless hosting za SvelteKit
 - Azure Portal → Static Web Apps → cloudsound-frontend
 - Custom domain configuration
 - Automatic SSL/TLS certificates
 2. **AKS container** (opcija 2) - frontend kot microservice v cluster-ju
 - Nginx serving static files
 - Exposed via Ingress Controller

Dostop:

- Production URL: `http://9.235.118.168/`
- API Gateway endpoint: `http://9.235.118.168/api/`

Development:

- Lokalno: `npm run dev` (dev server na `localhost:5173`)
 - Build: `npm run build` (production optimized bundle)
-

7.19 Terraform (Infrastructure as Code)

Projekt uporablja **Terraform** za deklarativno upravljanje Azure cloud infrastrukture. Terraform konfiguracija se nahaja v `infrastructure/terraform/` direktoriju. Definirani so moduli za: **AKS cluster** (node pools, scaling policies), **Azure Database for PostgreSQL Flexible Server** (HA configuration, backups), **Azure Storage Account** (blob containers za MP3 datoteke), **Azure Event Hubs** (Kafka replacement), **Virtual Network** (subnets, NSG rules), **Azure Container Registry** (Docker images), **Azure Key Vault** (secrets management). Spremenljivke so definirane v `variables.tf` z možnostjo override-a preko `terraform.tfvars` ali environment variables. State file je shranjen v Azure Storage Backend za team collaboration in state locking.

Azure Terraform workflow:

1. Terraform State Storage:

- Azure Portal → Storage Account → `tfstate-storage` → Containers → `terraform-state`
- State lock table: Azure Blob Storage lease mechanism

2. Deployment koraki:

```
cd infrastructure/terraform
terraform init # Initialize backend
terraform plan # Preview changes
terraform apply # Apply infrastructure
```

3. Managed resursi:

- Azure Portal → Resource groups → `cloudsound-rg`
- Vsi Terraform-managed resursi so tagged z `managed-by: terraform`
- Pregled: `terraform state list`

4. GitHub Actions integration:

- Workflow: `.github/workflows/terraform.yml`
- Automatic plan on PR, apply on merge to main
- Azure credentials stored in GitHub Secrets

Dostop do Terraform outputs:

```
terraform output # Prikaže AKS cluster name, PostgreSQL connection string, etc.
```

7.20 API Gateway

Mikrostoritev `cloudsound-api-gateway` (port 8000) deluje kot centralna vstopna točka (single entry point) za vse zunanje klice v sistem. Implementira več middleware komponent: **AuthMiddleware** za JWT avtentikacijo (validira access token, ekstrahira user identity in roles), **RateLimitMiddleware** za rate limiting per-IP ali per-user (npr. 100 requests/min) z uporabo Redis backend-a za distributed state, **CORS middleware** za cross-origin resource sharing policy, **ProxyMiddleware** za transparentno usmerjanje zahtev na backend servise glede na URL path (`/concerts/*` → `concert-management`, `/auth/*` → `authentication`). Gateway agregira service discovery informacije in load balancing preko Kubernetes Services.

Azure dostop:

- **API Gateway Service:** Deployiran kot Deployment v AKS
 - View: `kubectl get deployment api-gateway -n soundsound`
 - Logs: Azure Portal → AKS → Workloads → `api-gateway` → Logs
- **External Access:**
 - Ingress Controller → Load Balancer → Public IP: 9.235.118.168

- Azure Portal → AKS → Services and ingresses → cloudsound-ingress
- **Rate Limiting State:** Redis cluster v AKS
 - `kubectl port-forward svc/redis 6379:6379`
 - Monitor: `redis-cli MONITOR`

Testing:

```
curl -H "Authorization: Bearer $JWT_TOKEN" http://9.235.118.168/concerts/
```

7.21 Ingress Controller

Projekt uporablja **NGINX Ingress Controller** za upravljanje external access-a do Kubernetes Services. Ingress manifest definira routing rules: path-based routing (npr. `/concerts` → `concert-management-service`) in host-based routing (npr. `api.cloudsound.com`, `www.cloudsound.com`). NGINX anotacije omogočajo: SSL/TLS termination, rate limiting, request size limits, custom headers, URL rewriting, sticky sessions. Ingress Controller je deployiran kot DaemonSet oz. Deployment v AKS in izpostavljen preko Azure Load Balancer-ja z public IP naslovom. Support za Let's Encrypt automatic SSL certificates preko cert-manager addon-a.

Azure dostop:

- **Ingress Controller:**
 - Azure Portal → AKS → Services and ingresses → nginx-ingress-controller
 - Public IP: Static IP reserved v Azure
 - Load Balancer: Azure Portal → Load balancers → kubernetes-loadbalancer
- **Ingress Resources:**
 - `kubectl get ingress -n cloudsound`
 - Manifest: `infrastructure/helm/cloudsound/templates/ingress.yaml`
- **SSL Certificates:**
 - cert-manager za Let's Encrypt integracija (če je konfiguriran)
 - Secret: `kubectl get secret cloudsound-tls -n cloudsound`
- **Logs:**
 - NGINX access logs: `kubectl logs -n ingress-nginx deployment/ingress-nginx-controller`
 - Azure Monitor integration za centralized logs

Configuration: - Annotations v Ingress manifest-u za custom behavior - Example:

```
nginx.ingress.kubernetes.io/rate-limit: "100"
```

7.22 IAM, OAuth2, OIDC

Projekt uporablja **JWT (JSON Web Tokens)** avtentikacijo z **role-based access control (RBAC)** modelom. Implementacija je v `jwt_handler.py` modulu, ki podpira generiranje access tokens (short-lived, 15min) in refresh tokens (long-lived, 7 days). Tokeni vključujejo claims: `user_id`, `email`, `roles` (admin/user), `tenant_id`, `exp` (expiration), `iat` (issued at). AuthMiddleware v vseh servisih validira JWT signature z shared secret ključem in preverja token expiration. Admin endpoints so zaščiteni z `@require_role("admin")` dekoratorjem. Sistem **NE** uporablja zunanjih OAuth2 providers (Google, Facebook login) ali OIDC, ampak implementira lastno JWT-based avtentikacijo. Vsi passwords so hashed z bcrypt algoritmom z salt.

Azure security:

- **Key Vault za JWT Secret:**
 - Azure Portal → Key Vault → cloudsound-keyvault → Secrets → `jwt-secret-key`
 - Auto-rotation policy za enhanced security
- **Azure Active Directory Integration:**
 - Opcija za prihodnost: Integracija AAD za admin login
 - Managed Identities za service-to-service auth
- **Network Security:**
 - NSG (Network Security Groups) za firewall rules

- Private endpoints za PostgreSQL in Storage Account

Role management: - Roles shranjene v PostgreSQL users table - Admin creation: Manual SQL insert ali seeder script

Token validation: - Signature validation: HMAC SHA256 - Expiration check: Automatic rejection expired tokens - Blacklist: Redis-based token revocation list

8. Zaključek

CloudSound projekt predstavlja kompleksno mikrostoritveno aplikacijo za upravljanje koncertov in streaming glasbe, deployirano na Azure cloud platformi z uporabo moderne cloud-native arhitekture. Sistem demonstrira best practices za distributed systems: event-driven komunikacija, fault tolerance, observability, security in scalability. Uporaba Kubernetes orchestration, Infrastructure as Code (Terraform), CI/CD pipeline in comprehensive monitoring stack zagotavlja production-ready rešitev.

Tehnični dosežki:

- 8 neodvisnih mikrostoritev z jasno razdeljeno domeno odgovornosti
- Multi-repo Git organizacija z avtomatiziranim CI/CD pipeline-om
- Azure cloud deployment z Terraform IaC pristopom
- Prometheus + Grafana monitoring stack za 360° visibility
- Event-driven arhitektura z Kafka/Azure Event Hubs
- Fault-tolerant design s circuit breakers in retry logic
- Centralizirano beleženje z correlation ID tracing
- Serverless functions za compute-intensive operacije
- Multi-tenant ready arhitektura za prihodnje scaling

Prihodnje izboljšave:

- Implementacija GraphQL layer za bolj fleksibilne frontend queries
- OAuth2/OIDC integracija za social login
- Machine learning recommendations za personalizirane radijske postaje
- Real-time collaborative features (live chat med koncerti)
- Mobile aplikacija (React Native/Flutter)
- CDN integracija za globally distributed audio streaming
- Advanced analytics z AI-powered insights

Aplikacija je dostopna na <http://9.235.118.168/> in demonstrira uporabnost za realne use case-e glasbenega kluba.

Datum: Januar 2026

Avtorji: Tevž Sedmak, Matjaž Kumin

Projektna skupina: 23

GitHub: <https://github.com/CloudSound-MKNZ>